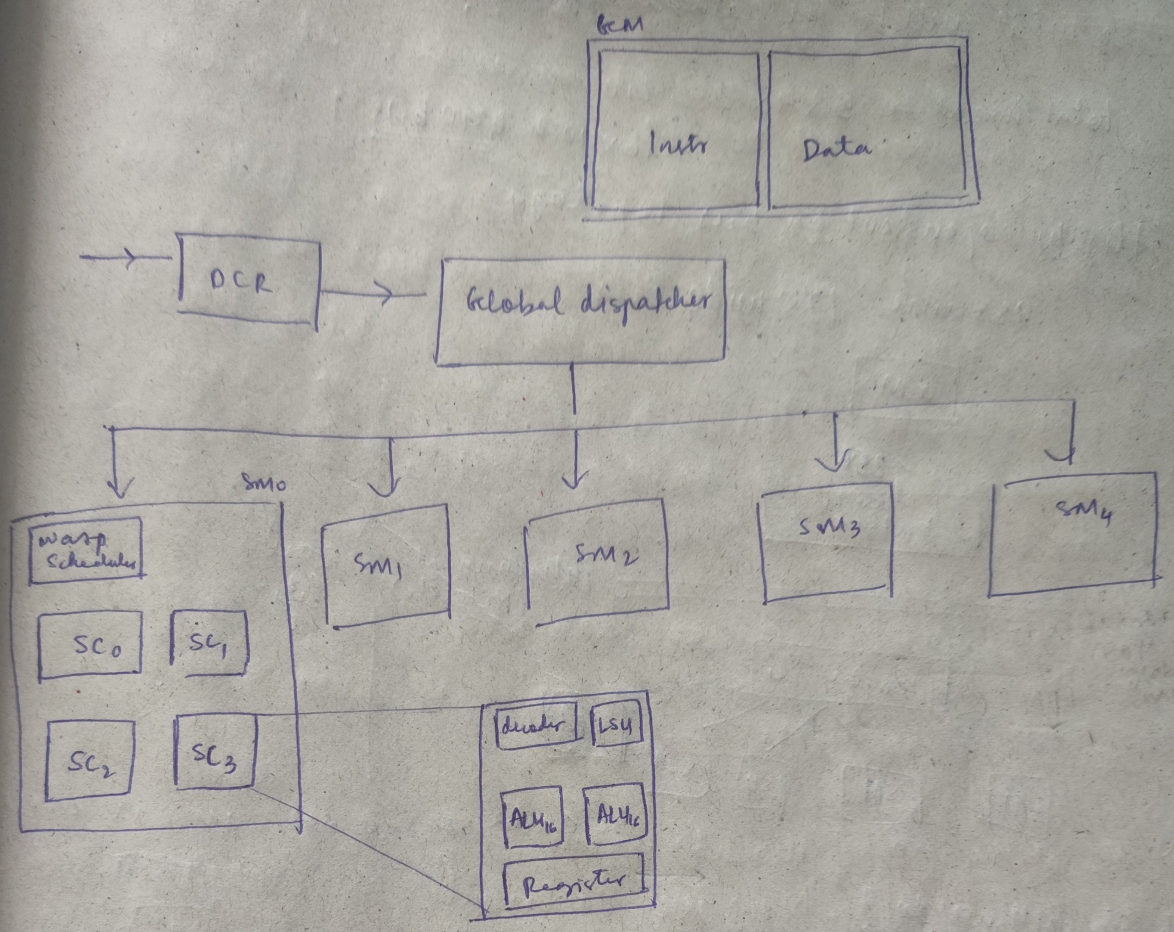
GPU-GPU



(1) Understanding the operation

in the GDM

LDR R2, R1

(a) R1 contains the pointer of the thread & R2 will actually hold the data

(b) Every subcore will run the same instr, so every subcore loads data when it sees this instr

Both R1 and R2 reside inside the subcores.

(c) LSU calculates effective address and stores it in the register. (*),

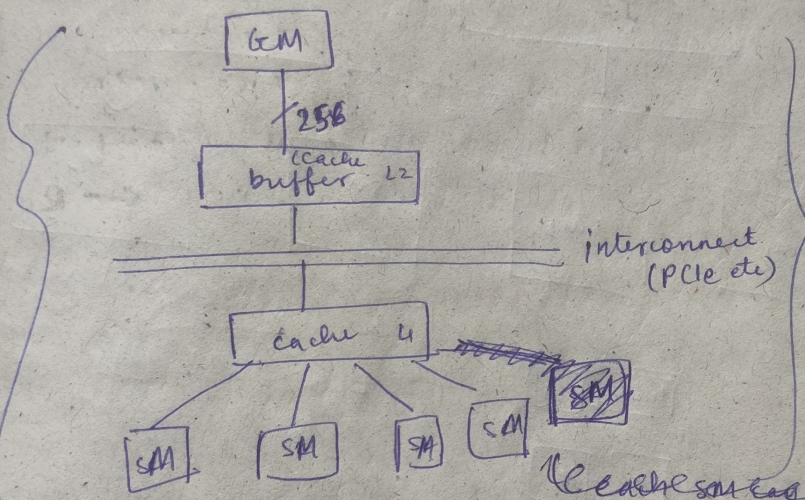
(2) We are fetching a lot of data at the same time so what will be the databus width?

total thread → 5 × 4 × 2 = 40

16 × 40 = 640 bits

Highly dependent on bus topology

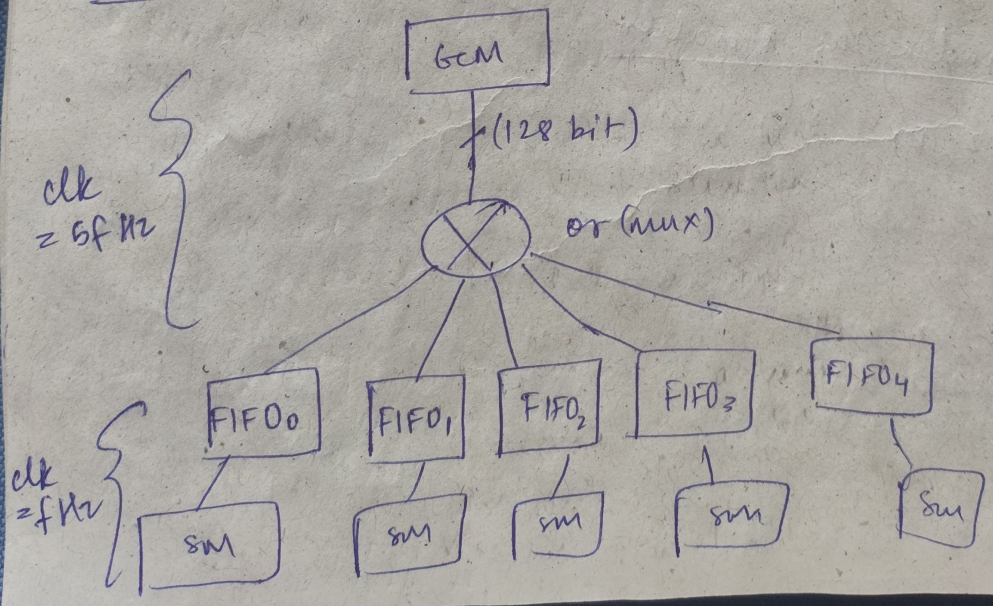
assume 128 wide bus,



~~Cache SM Cache data~~

~~40 × 2 = 8 × 16 = 128 bits~~

how will we do?



fifo writing speed = 5 times the FIFO reading speed.

③ For STR R_2, R_1 → gm[R2] ← R1 (data)

↙ just a register that contains the pointer of the GM, calculate the same way as in the LDR instruction.

Complete Flow

LDR	R3, R4	gm[R4] → R3
LDR	R1, R2	gm[R2] → R1
ADD	R3, R1	add R3+R1 → R3
STR	R5, R3	gm[R5] ← R3

(Register file will be dual port)
↳ will explain this ~~further~~ later (*) ||

Step A – (i) writing in DCR through tb. (we send a 64 bit ~~packet~~ tuple)

[Instr_ptr, No. of threads, I/p data ptr, O/p data ptr]
16 16 16 16

we will expand our architecture eventually such that even more than 40 threads can be processed (future work)
for now assume this value will be ≤ 40.

~~(#) return ptr get tuple start byte = 0~~

Step 2 : (i) from each SM, Global dispatcher will get a 5 bit tracking vector.
When, even one subcore is free, it will be 1
(only) when all the subcores are busy, it will be 0.

* As we are using round robin, a single SM will not get a lot of warps (we will distribute warps, not blocks). So let's say SM0 has only 1 free SC, and through RR, it get 4 warps, it means all others have got atleast 3. So these 3 will have to wait inside (Bottleneck)

* As our current dispatching algorithm is niche, we are having this issue, but we can easily resolve it in later stages.

(ii) We get the no. of threads from DCR, so we can calculate the no. of warps. Now we distribute the warps in RR.

* Remember we are still not dealing with data, so we are still not making the data travel. (It is just signals).

So, we send Work descriptor packet to each warp scheduler now.

(for sending WDP across each SM parallel same method as discussed before)

eg – 12 warps

SM0 – W_0, W_5, W_{10}
SM1 – W_1, W_6, W_{11}
SM2 – W_2, W_7, W_{12}
SM3 – W_3, W_8
SM4 – W_4, W_9

} assuming all subcores are free and no. of warps is ≤ 40

This initialization happens in all the registers in the subcore

→ PC_Init: Master PC = 0000h (1 PC for all sc)

→ the subcore_active mask gets updated

→ decode_flag turns high in the subcore, which should get the instr.

1st clock edge of mast_PC_SC → Instr is fetched from instr_mem into the SCB.

the Instr is sent to whatever subcores have (broadcasted) decode flag = 1, now, inside the decoder we will decode the instr how we usually do.

(As it is the ld instr, data should also be fetched from the data memory into the register file).

next Master PC value, ADD, and then finally str it into the Global memory. (Some signal goes high which is sent to DCR, indicating next DCR can be sent)

Case 2 — The "resource collision" problem

Let's say only 2 subcores were available, but we had 3 warps scheduled.

→ So we operate on the 2 warps and keep the 3rd one in queue. (turn some flag high)

→ In the 3rd master clk, data ~~gets~~ goes to the LSU, as the flag is indicating the incompleteness, we complete it and then send.

(4) Sending / Storing back data into the Global memory

In the 3rd master clk, data has to be stored back into the global memory. (Same Async FIFO method) here, write clk is at a lower freq that the read clk.

WDP

Warp stride (3)

[SM_ID (3), Warp_count (3), Base_Warp_ID (3), Instr_ptr (16), Data_ptr)

SM_ID — will be used as select

Warp_count — no. of warps that we are sending (000)

Base_Warp_ID — Src_ptr (16)

Instr_ptr (16)

Data_ptr — dst_ptr

80 bit

eg – {(000)$_b$, 101$_b$, (011)$_b$, 00000h, 0010h, 0100h}

* to do any operation, we would have to first load the data from the memory to registers, and for that, we have instr. LDR R1, R2.
How does R2 get the address (Src ptr) in the first place?

[AUTOMATIC HARDWARE REGISTER PRE-INIT] (*)III

Step 3: Inside the SM: (packet gets captured on the rising edge simultaneously in each SM)

(I) Reading WDP → dispatching it over subcores
→ pre-initializations.

(i) check status of the subcores (4 bit vector)

(ii) internal initialization.

case 1. (when all 4 subcores are available)

→ load the src ptr address into a default register. Same for the dst_ptr.

Src_ptr + Warp_no. × warp stride + base_warpID

(5) What are all the clock operating in this architecture?

→ only 1

1st clk pulse → DCR activation

2nd clk pulse → Global dispatcher

3rd clk pulse → Warp scheduler gets WOP

4th clk pulse → PC = 0 execution starts

⋮

until the previously described signal again goes high, its stays in the 4th state, clk keeps running finally